Does a Learned Tokenizer Help a Small DNA Language Model? GBST-BERT on the Human Genome

Omar Mohamed (K12103822)
Practical Work, Institute for Machine Learning
Johannes Kepler University Linz, Austria

Abstract

DNA language models usually fix their tokenization in advance. ChaRNABERT showed for RNA that a learned tokenizer, Gradient-Based Subword Tokenization (GBST), can replace this choice. We implement this architecture in the DNA-xLSTM code base and pretrain a 2.1M-parameter model, GBST-BERT, on the human genome under the short-context protocol of Bio-xLSTM, next to a Transformer++ control of the same shape. Pretraining does not separate the two models (test accuracy on masked bases 56.44% vs. 56.33%). On the 18 Nucleotide Transformer tasks, GBST-BERT has the best mean of seven models on 13 tasks and exceeds the control on 16, with the largest gains on histone marks. On the 8 Genomic Benchmarks tasks the advantage is small (higher mean on 5 of 8). GBST places 98% of its weight on single bases, so it does not learn multi-base units on human DNA; we suspect the gain comes from its convolution, but the control also differs in normalization and learning-rate tuning, so this remains to be tested. Finally, we show that the weak enhancer scores coincide with a train–test mismatch in the benchmark itself: a plain 4-mer classifier shows the same drop from validation to test, and a quarter of the test sequences are exact copies of training sequences.

1 Introduction

A language model must first decide how to cut its input into units. For DNA this choice is not obvious: single bases keep full resolution, while k-mers or byte-pair encodings shorten the sequence but fix the units in advance and can shift a motif across token boundaries. ChaRNABERT [Morales-Pastor et al., 2024] adapts GBST from Charformer [Tay et al., 2022] so that a model learns, at every position, how to combine neighbouring characters, and reports benefits on RNA. Whether this transfers to DNA, at the small scale where DNA-xLSTM [Schmidinger et al., 2024] reports strong results, is the question of this practical work.

We keep the architecture and change the data: GBST and a bidirectional encoder, trained on hg38 with masked language modelling (MLM) and evaluated with the protocols of Bio-xLSTM. Our contributions are:

- an implementation of GBST-BERT in the DNA-xLSTM code base, tested against a reference implementation of the published equations and released with a one-file model and a notebook;
- a controlled comparison with Transformer++ in pretraining and on 26 downstream tasks (464 fine-tuning runs), reported with all repeats;
- an analysis of what GBST learned on DNA, which differs from the motivation for using it;

Preprint.

- a diagnosis of the enhancer tasks, which affects how these two benchmark tasks should be read.

2 Background

Tokenization of biological sequences. Bio-xLSTM and Caduceus [Schiff et al., 2024] use one token per base, as does HyenaDNA [Nguyen et al., 2023]; the Nucleotide Transformer [Dalla-Torre et al., 2023] uses k-mers. GBST [Tay et al., 2022] forms candidate blocks of characters and lets a scoring network weight them, which makes the tokenization differentiable. ChaRNABERT removes Charformer's downsampling, so that one output per character remains.

Evaluation. Bio-xLSTM evaluates DNA models of 2M parameters on the 18 Nucleotide Transformer (NT) tasks, and of 500k parameters on the 8 Genomic Benchmarks tasks [Grešová et al., 2023]. We reuse its reported values as references.

3 Method

GBST-BERT. Let $x_1, \ldots, x_L$ be the bases of a sequence. Each is embedded in $d = 256$ dimensions; a depthwise convolution of width 4 and a pointwise $d \times d$ layer mix each position with its neighbours. For each position i, GBST forms every block of size $b \in \{1, 2, 3, 4\}$ that contains i (one block per offset, $K = 10$ candidates), and pools each block to a vector $h_{i,k}$. A linear scorer gives one score per candidate, and a softmax over candidates gives weights $P \in \mathbb{R}^{L \times K}$. Following ChaRNABERT, the weights are calibrated across positions, $\hat{P} = \mathrm{softmax}(PP^\top)P$, and position i receives the mix $\tilde{x}_i = \sum_k \hat{P}_{ik}\, h_{i,k}$. There is no downsampling. The encoder has 4 bidirectional Transformer blocks with 8 heads, RoPE [Su et al., 2024], QK normalization and SwiGLU [Shazeer, 2020] (hidden size 320), followed by an MLM head. The model has 2,107,392 parameters, of which 70,144 belong to GBST.

Control. Transformer++-2M [Schmidinger et al., 2024, Table A3] has the same width, depth and heads and a plain embedding table (2,036,992 parameters). It is a close control but not a perfect one: it uses RMSNorm where GBST-BERT uses LayerNorm and QK normalization, and its learning rate is the paper's value, whereas GBST-BERT's was chosen by a short search (Section 4).

4 Experimental setup

Pretraining. We follow the short-context protocol of Bio-xLSTM: hg38, pieces of 1,024 bases, 2^{20} bases per step, 10,000 steps (10.5B bases), reverse-complement augmentation, and BERT-style masking [Devlin et al., 2019] of 15% of positions. For GBST-BERT, learning rates $\{5\mathrm{e}{-4}, 1\mathrm{e}{-3}, 2\mathrm{e}{-3}, 5\mathrm{e}{-3}\}$ were compared over 500 steps by validation loss, and $2\mathrm{e}{-3}$ was used; Transformer++ keeps the paper's $1\mathrm{e}{-2}$. Each model was pretrained once. Appendix A lists all settings.

NT benchmark. Following Schmidinger et al. [2024, App. C.5]: mean pooling and a linear head, full fine-tuning for 20 epochs, testing the epoch with the best validation score, 10 seeds with a new 90/10 train/validation split each. We use the post-hoc conjoining (PH) setting: reverse-complement augmentation in training, and predictions averaged over both strands at test time [Schiff et al., 2024]. Learning rate and batch size per task are those Bio-xLSTM selected for DNA-xLSTM-PH (its Table A5). On splice acceptors and donors these values made GBST-BERT collapse to one class (F1 0.663 in all first three donor runs), so $1\mathrm{e}{-3}$ and $5\mathrm{e}{-4}$ were also run for both models with 3 seeds, and the setting with the best mean *validation* score was completed to 10 seeds.

Genomic Benchmarks. Same procedure, with 10 epochs, 5 seeds and top-1 accuracy, as in the paper. It reports no per-task settings for this benchmark, so all tasks use one setting from its grid (learning rate $1\mathrm{e}{-3}$, batch 128). Sequences are cut to 1,024 bases.

Reporting. As in Bio-xLSTM, we give the mean $\pm$ the range (maximum minus minimum) over seeds. Means within 0.0005 count as ties. The test sets were never used to choose a setting.

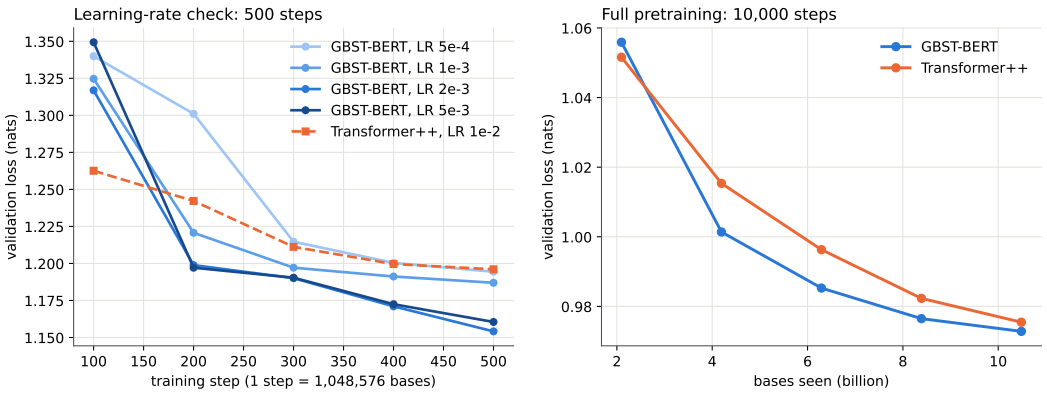


Figure 1: Validation loss. Left: learning-rate search for GBST-BERT over 500 steps, with Transformer++ at its paper value. Right: full pretraining of both models.

Table 1: Masked-base prediction on 2,048 unseen test sequences (311,767 masked bases, identical masks for both models).

Model	Test loss (nats)	Accuracy
GBST-BERT	**0.9753**	**56.44%**
Transformer++	0.9774	56.33%
Random guess	1.3863	25.00%
Always the most common base	–	29.42%

5 Results

5.1 Pretraining

Both models learn well beyond base composition (Table 1). GBST-BERT is ahead on validation loss for most of training (Figure 1), but the final gap of 0.11 percentage points cannot be interpreted with one run per model. Pretraining does not separate them.

5.2 What GBST learned

We measure the share of GBST's weight on each block size, averaged over 8 hg38 sequences from different chromosomes. For blocks of 1/2/3/4 bases it is 72/22/5/1% after 500 steps at learning rate $5e{-}4$, but 98/2/0/0% after 500 steps at $2e{-}3$ and in the final model. GBST therefore does *not* learn multi-base units on human DNA under this setup: at the learning rate we selected, it reads single bases almost exclusively. Each base has already been mixed with its neighbours by the convolution, so the module acts mainly as a learned local filter. The early difference between learning rates also shows that this outcome depends on training conditions.

5.3 Nucleotide Transformer benchmark

GBST-BERT has the best mean on 13 of 18 tasks (Table 2; xLSTM-PH 4, Mamba-PH 2, Mamba-PS 1, Transformer++ 0). It exceeds Transformer++ on 16 of 18 tasks (two-sided sign test over tasks $p = 0.0013$; the tasks are related, so this value is only a guide). The largest gains are on histone marks (e.g. H3K4me3 $+0.190$, H3K4me2 $+0.142$, H4ac $+0.114$ MCC) and on splice donors ($+0.071$ F1), while promoters differ by at most 0.005. Compared with the best paper model per task, GBST-BERT is higher on 12 tasks and tied on one. The splice tasks received an extra learning-rate check, which the paper's models did not need. Some GBST-BERT ranges are wide (H3K4me2 ± 0.212, H3K4me3 ± 0.180), so single runs are unreliable on these tasks.

Table 2: NT benchmark test scores, mean ± range over 10 seeds. The first five columns are Bio-xLSTM Table 1; the last two are ours. Bold: best mean (ties counted for each model).

Task	HyenaDNA	Mamba-PS	Mamba-PH	xLSTM-PS	xLSTM-PH	GBST-BERT	Transformer++
Histone marks (MCC)							
H3	0.779 ± 0.037	0.799 ± 0.029	0.815 ± 0.048	0.796 ± 0.014	$\mathbf{0.824 \pm 0.010}$	0.815 ± 0.031	0.801 ± 0.029
H3K14ac	0.612 ± 0.065	0.541 ± 0.212	0.631 ± 0.026	0.570 ± 0.008	0.598 ± 0.017	$\mathbf{0.673 \pm 0.016}$	0.569 ± 0.062
H3K36me3	0.613 ± 0.041	0.609 ± 0.109	0.601 ± 0.129	0.588 ± 0.019	0.625 ± 0.010	$\mathbf{0.655 \pm 0.035}$	0.591 ± 0.032
H3K4me1	0.512 ± 0.024	0.488 ± 0.102	0.523 ± 0.039	0.490 ± 0.012	0.526 ± 0.001	$\mathbf{0.565 \pm 0.038}$	0.481 ± 0.037
H3K4me2	0.455 ± 0.095	0.388 ± 0.101	0.487 ± 0.170	0.489 ± 0.024	0.504 ± 0.012	$\mathbf{0.509 \pm 0.212}$	0.367 ± 0.076
H3K4me3	0.549 ± 0.056	0.440 ± 0.202	0.544 ± 0.045	0.520 ± 0.019	0.537 ± 0.012	$\mathbf{0.553 \pm 0.180}$	0.362 ± 0.219
H3K79me3	0.672 ± 0.048	0.676 ± 0.026	$\mathbf{0.697 \pm 0.077}$	0.662 ± 0.011	$\mathbf{0.697 \pm 0.007}$	$\mathbf{0.697 \pm 0.132}$	0.676 ± 0.041
H3K9ac	0.581 ± 0.061	0.604 ± 0.048	0.622 ± 0.030	0.622 ± 0.013	0.627 ± 0.008	$\mathbf{0.648 \pm 0.037}$	0.599 ± 0.046
H4	0.763 ± 0.044	0.789 ± 0.020	0.811 ± 0.022	0.793 ± 0.011	$\mathbf{0.813 \pm 0.008}$	0.804 ± 0.074	0.803 ± 0.022
H4ac	0.564 ± 0.038	0.525 ± 0.240	0.621 ± 0.054	0.558 ± 0.018	0.583 ± 0.014	$\mathbf{0.642 \pm 0.037}$	0.528 ± 0.036
Enhancers (MCC)							
enhancers	0.517 ± 0.117	0.491 ± 0.066	$\mathbf{0.546 \pm 0.073}$	0.375 ± 0.030	0.545 ± 0.024	0.336 ± 0.085	0.368 ± 0.074
enhancers_types	0.386 ± 0.185	0.416 ± 0.095	0.439 ± 0.054	0.444 ± 0.046	$\mathbf{0.466 \pm 0.011}$	0.430 ± 0.167	0.384 ± 0.053
Promoters (F1)							
promoter_all	0.960 ± 0.005	0.967 ± 0.004	0.970 ± 0.004	0.962 ± 0.002	0.967 ± 0.001	$\mathbf{0.971 \pm 0.003}$	0.968 ± 0.006
promoter_no_tata	0.959 ± 0.011	0.968 ± 0.006	0.968 ± 0.010	0.963 ± 0.002	0.970 ± 0.001	$\mathbf{0.972 \pm 0.006}$	0.969 ± 0.004
promoter_tata	0.944 ± 0.040	$\mathbf{0.957 \pm 0.015}$	0.953 ± 0.016	0.948 ± 0.006	0.952 ± 0.005	0.948 ± 0.021	0.953 ± 0.013
Splice sites (accuracy / F1)							
splice_sites_all	0.956 ± 0.011	0.927 ± 0.021	0.940 ± 0.027	0.965 ± 0.006	0.974 ± 0.004	$\mathbf{0.980 \pm 0.004}$	0.975 ± 0.006
splice_sites_acceptors	0.958 ± 0.010	0.936 ± 0.077	0.937 ± 0.033	0.970 ± 0.005	0.953 ± 0.008	$\mathbf{0.977 \pm 0.009}$	0.955 ± 0.049
splice_sites_donors	0.949 ± 0.024	0.874 ± 0.289	0.948 ± 0.025	0.962 ± 0.004	0.951 ± 0.005	$\mathbf{0.980 \pm 0.012}$	0.909 ± 0.117
Tasks with the best mean	0	1	2	0	4	13	0

Table 3: Validation and test scores (MCC). Our models: mean over 10 seeds. 4-mer baseline: logistic regression on 4-mer frequencies, one 90/10 split. Last column: test sequences with an exact copy in the training file. Copies always carry the same label.

	GBST-BERT		Transformer++		4-mer logistic regression			Test seqs.
Task	val.	test	val.	test	val.	test	test, no copies	in train
enhancers	0.871	0.336	0.899	0.368	0.794	0.436	0.319	100/400
enhancers_types	0.789	0.430	0.788	0.384	0.777	0.299	0.168	100/400
promoter_all	0.973	0.971	0.970	0.968	0.807	0.802	0.802	21/5920
H3	0.811	0.815	0.787	0.801	0.634	0.631	0.631	0/1497

5.4 Why the enhancer tasks are weak

Enhancers are the clear weakness: MCC 0.336 against 0.546 for the best paper model. Our logs show that these two tasks stand apart: validation scores far exceed test scores (gap 0.53 and 0.36 for GBST-BERT), while on the 16 other tasks the gap never exceeds 0.05. Transformer++ behaves the same way, so GBST is not the cause.

To test whether this is a modelling problem at all, we trained a plain logistic regression on 4-mer frequencies, with no neural network and no pretraining (Table 3). It shows the same drop on the enhancer tasks (0.79 to 0.44, and 0.78 to 0.30) and none on the controls. The gap is therefore a property of the dataset: what separates the classes in the training file transfers only partly to the test file. On the binary task this simple baseline even exceeds both of our models on test, which suggests that they fit training-specific patterns more closely than a 4-mer model can.

Two further points affect how these tasks should be read. First, 100 of the 400 test sequences are exact copies of training sequences, so the test score partly measures recall; the baseline's score falls further when they are removed. Second, validation scores cannot guide model selection here, as they do not reflect test behaviour. We did not determine why some published models reach higher test scores on these tasks; with 400 test sequences the estimates are also noisy.

5.5 Genomic Benchmarks

Here the picture is much weaker (Table 4). GBST-BERT has the higher mean than Transformer++ on 5 of 8 tasks (sign test $p = 0.73$), by at most 0.8 percentage points. It is ahead on all 5 splits on coding

Table 4: Genomic Benchmarks test accuracy, mean ± range over 5 seeds. Paper models have 500k parameters and ours 2M, so the paper's values are a reference only. Last column: splits on which GBST-BERT is above Transformer++.

Task	Bio-xLSTM Table A2 (500k)				Ours (2M)		GBST > T++
	Mamba-PH	xLSTM-PH	Mamba-PS	xLSTM-PS	GBST-BERT	Transformer++	(splits)
Mouse enhancers	0.754 ± 0.074	0.780 ± 0.018	$\mathbf{0.793 \pm 0.058}$	0.778 ± 0.007	0.761 ± 0.116	0.776 ± 0.041	4/5
Coding vs. intergenomic	0.915 ± 0.003	0.931 ± 0.001	0.910 ± 0.003	0.934 ± 0.002	$\mathbf{0.937 \pm 0.002}$	0.930 ± 0.007	5/5
Human vs. worm	$\mathbf{0.973 \pm 0.001}$	0.965 ± 0.001	0.968 ± 0.002	0.956 ± 0.001	0.971 ± 0.003	0.972 ± 0.002	1/5
Human enhancers (Cohn)	$\mathbf{0.747 \pm 0.004}$	0.742 ± 0.005	0.745 ± 0.007	0.734 ± 0.005	0.738 ± 0.019	0.736 ± 0.014	3/5
Human enhancers (Ensembl)	0.893 ± 0.008	$\mathbf{0.920 \pm 0.001}$	0.900 ± 0.006	0.902 ± 0.004	0.903 ± 0.008	0.905 ± 0.008	1/5
Human regulatory	0.872 ± 0.011	0.872 ± 0.002	0.873 ± 0.007	0.869 ± 0.005	$\mathbf{0.885 \pm 0.003}$	0.883 ± 0.004	4/5
Human open chromatin	0.828 ± 0.006	0.826 ± 0.002	0.818 ± 0.006	0.800 ± 0.002	$\mathbf{0.841 \pm 0.009}$	0.836 ± 0.008	5/5
Human non-TATA promoters	0.946 ± 0.007	0.951 ± 0.004	0.945 ± 0.010	0.949 ± 0.001	$\mathbf{0.955 \pm 0.015}$	0.952 ± 0.004	2/5
Tasks with the best mean	2	1	1	0	4	0	

vs. intergenomic and open chromatin, and behind on 4 of 5 splits on two tasks. Mouse enhancers (1,210 sequences, 97 for validation) shows a lower mean although GBST-BERT wins 4 of 5 splits: in one split an early epoch scored highest on validation and then reached only 0.68 on test. Overall this benchmark does not show a clear benefit from GBST. It uses larger datasets, up to 289k sequences, and one untuned setting, and we cannot separate these factors.

6 Discussion and limitations

What can be claimed. GBST-BERT is a competitive small DNA model: on the NT benchmark it matches or exceeds published 2M models on most tasks. It improves on its control on the NT benchmark, clearly on histone marks, and only marginally on Genomic Benchmarks.

What cannot be claimed yet. We have not shown that *tokenization* causes the gain. GBST reads single bases, so learned multi-base units cannot explain it; we suspect the convolution, since local context suits motif-driven tasks, but this is a hypothesis. In addition, the two models differ in normalization and in how their pretraining learning rate was chosen. The decisive experiment is GBST-BERT with GBST replaced by an embedding table, under otherwise identical settings. The code supports it (`TOKENIZER=embedding`) but it was not run. A further variant keeping only the convolution would isolate its contribution.

Further limitations. Each model was pretrained once, so variation from pretraining is unmeasured and the fine-tuning repeats capture only part of the uncertainty. Fine-tuning settings came from a different architecture (DNA-xLSTM-PH) and were not tuned for ours, except on splice tasks. GBST-BERT's first 3,000 pretraining steps used a data loader with a bug that corrupted 0.35% of sequences (Appendix B). We tested only one scale and context length, and block sizes up to 4. Our results concern classification benchmarks, not any clinical use.

7 Conclusion

A learned tokenizer did not behave as motivated on human DNA: it chose single bases. Even so, the resulting model is competitive at 2M parameters and beats its control on most NT tasks, with a much smaller effect on a second benchmark. The next steps are the ablation above and a repeat of pretraining with other seeds. Separately, the two NT enhancer tasks should be interpreted with care, given their train–test mismatch and duplicated sequences.

Code and reproducibility

- Repository (branch `charna-gbst`):
  https://github.com/OMAR622-ai/dna-xlstm-charna
- Full notebook, executed, with every step, table and figure:
  https://github.com/OMAR622-ai/dna-xlstm-charna/blob/charna-gbst/notebooks/GBST_BERT_full_pipeline.ipynb

- Fine-tuned NT models:
  https://github.com/OMAR622-ai/dna-xlstm-charna/releases/tag/nt-finetuned-models

The notebook re-computes the data statistics, the final test of both pretrained models, the GBST analysis, all tables and figures, the enhancer diagnosis, and one fine-tuning job per benchmark. It loads the saved results of the original pretraining and fine-tuning runs; setting its switches to `True` repeats those long runs with the same scripts.

References

Hugo Dalla-Torre, Liam Gonzalez, Javier Mendoza-Revilla, Nicolas Lopez Carranza, Adam Henryk Grzywaczewski, Francesco Oteri, Christian Dallago, Evan Trop, Bernardo P. de Almeida, Hassan Sirelkhatim, et al. The nucleotide transformer: Building and evaluating robust foundation models for human genomics. *bioRxiv*, pages 2023–01, 2023.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*, pages 4171–4186, 2019.

Katarína Grešová, Vlastimil Martinek, David Čechák, Petr Šimeček, and Panagiotis Alexiou. Genomic benchmarks: a collection of datasets for genomic sequence classification. *BMC Genomic Data*, 24(1):25, 2023.

Adrián Morales-Pastor, Raquel Vázquez-Reza, Miłosz Wieczór, Clàudia Valverde, Manel Gil-Sorribes, Bertran Miquel-Oliver, Álvaro Ciudad, and Alexis Molina. Character-level tokenizations as powerful inductive biases for RNA foundational models. *arXiv preprint arXiv:2411.11808*, 2024.

Eric Nguyen, Michael Poli, Marjan Faizi, Armin Thomas, Michael Wornow, Callum Birch-Sykes, Stefano Massaroli, Aman Patel, Clayton Rabideau, Yoshua Bengio, et al. HyenaDNA: Long-range genomic sequence modeling at single nucleotide resolution. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 43177–43201, 2023.

Yair Schiff, Chia Hsiang Kao, Aaron Gokaslan, Tri Dao, Albert Gu, and Volodymyr Kuleshov. Caduceus: Bi-directional equivariant long-range DNA sequence modeling. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235, pages 43632–43648, 2024.

Niklas Schmidinger, Lisa Schneckenreiter, Philipp Seidl, Johannes Schimunek, Pieter-Jan Hoedt, Johannes Brandstetter, Andreas Mayr, Sohvi Luukkonen, Sepp Hochreiter, and Günter Klambauer. Bio-xLSTM: Generative modeling, representation and in-context learning of biological and chemical sequences. *arXiv preprint arXiv:2411.04165*, 2024.

Noam Shazeer. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

Yi Tay, Vinh Q. Tran, Sebastian Ruder, Jai Gupta, Hyung Won Chung, Dara Bahri, Zhen Qin, Simon Baumgartner, Cong Yu, and Donald Metzler. Charformer: Fast character transformers via gradient-based subword tokenization. In *International Conference on Learning Representations (ICLR)*, 2022.

A Settings

Table 5 lists the main settings and their sources.

Table 5: Main settings and their sources.

Model	256 dimensions, 4 blocks, 8 heads, SwiGLU hidden size 320, no norm or linear biases [Bio-xLSTM Table A3]
GBST	blocks of 1–4 bases with offsets, sum pooling, calibration, no downsampling [ChaRNABERT Sec. 3.2]; maximum block size 4 is our choice
Masking	15% of positions; 80% `[MASK]`, 10% random, 10% unchanged [Bio-xLSTM App. C.5]
Batch	1,024 sequences of 1,024 bases per step [Bio-xLSTM App. C.5]
Optimizer	AdamW, $\beta = (0.9, 0.95)$, weight decay 0.1, 10% warm-up, cosine decay [Bio-xLSTM Table A4]
Learning rate	GBST-BERT $2\mathrm{e}{-3}$ (our search); Transformer++ $1\mathrm{e}{-2}$ [Bio-xLSTM Table A3]
Precision	32-bit; our GPUs (GTX 1080 Ti, RTX 2080 Ti, Quadro GV100) have no bf16

B Differences from the papers and known issues

- **ChaRNABERT Eq. 1** describes a depthwise convolution, but its Table 9 counts $d \cdot M + d^2$ weights, which implies an additional pointwise layer. We include it.
- **ChaRNABERT Eq. 3** read literally assigns a position blocks that do not contain it. We use blocks containing the position, as the text on offsets describes.
- **ChaRNABERT Eq. 9** differs from standard SwiGLU, which the text cites. We use the standard form.
- **Attention.** We use PyTorch's scaled dot-product attention instead of FlashAttention 2, which our GPUs do not support; the attention computed is the same.
- **NT data loader.** The repository's loader mixed all 18 tasks after a change on Hugging Face (461,850 examples instead of e.g. 5,509); it now loads each task's own file.
- **Genome reader.** Parallel reader processes shared one open file and sometimes read wrong bases: 0.35% of sequences with 2 workers per GPU, as used in the first part of training. Wrong reads stayed inside their data split. After the fix, GBST-BERT resumed from step 3,000 and Transformer++ was trained from scratch, so only GBST-BERT saw affected data.
- **Validation at step 10,000.** After resuming, GBST-BERT's counter ran one step ahead and skipped this check; it was measured afterwards on the final model with the same settings.